Certified Multi-Reflex Chains at Million Scale: Toward Low-Energy, Skill-Cohesive AI Scientific Solvers

Kavan Shah

Adaptive Reflex Engine Project (V1.7 — the nineteenth pre-registered experiment of the arc)

Technical report, September 4, 2026 · repo `reflex-engine`, tag `v1.7-final` (819115d) · not peer-reviewed · all sealed datasets hashed at pre-registration and evaluated once

Abstract

Large language models answer scientific questions with high coverage but nonzero, unpredictable error, at an energy cost of joules to kilojoules per query. We study the opposite design point: an engine that serves an answer only when two independent implementations of a typed, auditable computation agree, and abstains otherwise — with frontier LLMs used exclusively to propose *procedures and structure*, never answers. V1.7 tests whether such certified skills (*reflexes*) **compose**: whether chains of two to six reflexes can be served deterministically, whether four frontier models (claude-haiku, -sonnet, -opus, -fable) can plan novel chains that pass certification, how long structure-finding takes as a function of chain depth, and what happens when certified plans are *reused* as templates at million scale. On a pre-registered, hash-sealed corpus of 1,000,000 generated chain questions we report: (i) a deterministic composer that served 112,170 sealed chain answers with **zero errors**, covering 30.4% of questions requiring 3+ chained steps; (ii) model-planned structures certified at 93.2–95.8% for the three larger models with **zero wrong certified answers in 3,600 sealed plans** (the smallest model produced 13 wrong in 1,200, all one mechanism); (iii) structure-finding time of 4.6–6.5 s that is **near-flat in chain depth** (+10–19% from 3-step to 6-step); (iv) a strictly monotone scale gradient — coverage up, all error measures down — across the four 1M-item arms; and (v) a sharp negative result: positional template reuse produced 8,842 wrong answers across 4M evaluations (0.24–0.80% of serves per arm), every one traced to role-blind number binding, while the role-aware composer inside identical runs made zero errors. We argue this cleanly reduces safe skill reuse to role-aware binding, and we situate the result in the programme's measured energy trajectory: 21 mJ per certified serve versus 1,910 J for a measured local LLM baseline — a ~90,000x serving-energy gap that motivates the programme.

1 Introduction and goal

The goal of this programme is an **AI scientific solver with two properties current LLM deployments lack**:

- **Radically lower serving energy.** A solver that answers most routine scientific/quantitative questions through certified deterministic procedures at millijoule cost, escalating to a large model only for genuinely novel structure. In our measurements a certified reflex serve costs ~21 mJ and ~1–7 ms; a measured local-LLM baseline (llama-3.1-8B, instrumented wall-power) costs 1,910 J per query (Section 8). If a growing skill library absorbs a rising share of traffic, fleet energy scales with the *residual*, not the total.
- **Skill cohesion.** Skills must not be isolated party tricks: they must be indexed (V1.6 introduced a 47-leaf taxonomy), grow by model-proposed acquisition under an admission gauntlet, and — the subject of this report — **compose into chains** so that learning differentiation, unit conversion, and compound interest separately yields the ability to answer a question that needs all three in sequence.

The safety invariant that governs every design choice is **UNKNOWN > WRONG**: an abstention costs a fallback to a slower path; a confidently wrong answer contaminates everything downstream. Concretely, nothing is ever served unless a static plan gate passes *and* two independent implementations (sympy- and mpmath-based, sharing no code path) agree on every step and the final value. Frontier models participate only as *proposers* — of parse procedures, chain reflexes, and plan structures — which face certification before a single serve.

V1.7 is the nineteenth experiment of an arc (V0.1–V1.6) that has alternated capability gains with pre-registered negative results. Section 2 states the methodology; Section 3 compresses the arc; Sections 4–7 present V1.7's system and results; Section 8 gives the energy position with exact measured values; Section 9 analyses the failures; Sections 10–11 discuss limitations and the road forward.

2 Methodology: how every number below is protected

- **Pre-registration.** Hypotheses, metrics, thresholds, and a six-letter outcome ladder (A best – F worst, priority F>E>D>C>B>A) are frozen in the spec (`V17_SPEC.md`, commit b87ff8f) before any sealed item is touched. The version's outcome letter is decided mechanically by the frozen text.
- **Hash-sealed, run-once evaluation.** The corpus is committed with raw-byte SHA-256 hashes at freeze; sealed splits are evaluated exactly once per pre-registered arm; development and sealed data never mix. (A V1.5 protocol slip — development against a sealed set, caught mid-build — was disclosed in that report and led to stricter splits.)
- **Numbered bug audit.** Every defect receives a numbered entry (B1–B32 across the arc) with mechanism, fix, and verification; superseded runs are preserved in-repo, and each wrong answer is committed as a traceable row (id, text, served value, gold, serving path).
- **Instrumentation discipline.** Model self-reports are never used for time or energy (audit entry B7). V1.7 timing derives from harness-observed file-modification timestamps; energy figures cited from earlier versions were wall-power measured with warm-up exclusion and positive controls (a lesson of B24, where a meter reading 0.0 J everywhere was caught by exactly such a control).
- **Positive controls everywhere.** Every detector, gate, and witness must first fail a battery of deliberately wrong inputs (one sound plan + eight wrong-plan classes for the V1.7 gate; sabotage batteries for the witness). A green check that cannot go red has bitten this arc five times; it is now a standing rule.

3 Eighteen prior experiments in brief

Figure 1 shows each version's pre-registered outcome and defining result. Three threads matter for this report: the **composition thread** (V0.9's planner died at 5–7 steps, 0/25, while shallow paths were 125/125; V1.5 measured 40% wrong-fire when composition was forced, coverage 0.63%); the **safety thread** (V1.2's formal certificate: 10,160 adversarial traps x 65 seeds at 0.000 wrong-fire; V1.6: 118k+ sealed serves, zero wrong); and the **learning thread** (V1.0 proved the acquisition channel: model-mined procedures added +7.2pp certified coverage at 0.000 wrong-fire; V1.6 grew a 271-reflex library that lifted foreign-phrased coverage 35.4%→55.5%).

The arc at a glance: pre-registered outcome letters (A best - F worst) and each version's defining result

A	**V0.5**	574 vs 1687 J/correct
E	**V0.7**	recognition wall
C	**V0.8**	typed boundary = the mechanism
F	**V0.9**	depth wall at 5-7 steps
F	**V0.10**	196/196 at 0 LLM calls; gate excluded all
A	**V0.11**	0.000 wrong-exec everywhere
C	**V1.0**	acquisition channel proven (+7.2pp)
A	**V1.1**	generator universe closed; 95.5% @ 0 wrong
B	**V1.2**	certified 0.000 over 10,160 traps / 65 seeds
D	**V1.3**	real language read: 209 certified / 3 wrong
D	**V1.4**	state substrate safe but inert (0/240 certified)
E	**V1.5**	composition is the bottleneck
B	**V1.6**	foreign coverage 35.4 -> 55.5% @ 0 wrong
E	**V1.7**	chains compose: 30.4% @ 0 wrong; reuse unsafe

Figure 1: The arc V0.5–V1.7 — pre-registered outcome letters and defining results. E/F letters are safety or premise failures; the ladder deliberately ranks any safety breach above any capability win.

Two prior sentences frame V1.7. From V1.3 (plan reflexes falsified, 7 served / 7 wrong): *“checks verify that a plan matches the TEXT, never that it answers the QUESTION.”* And from V1.5: *“knowing a story contains 47, 41 and 20 is not knowing the answer is 47–41+20.”* V1.7's architecture is built to beat both — and its one surviving failure mode (Section 9) is precisely their echo in the reuse layer.

4 The V1.7 system

4.1 Plan representation

A *plan* is a typed DAG serialized as JSON. Node operations: `evaluate` (arithmetic over earlier outputs), `formula` (physics/finance expressions with named slots and unit annotations), `scale`, `convert` (pint-backed), `derivative_at`, `integrate_def`, `solve_pick`, `sum_finite`, `gcd`, `lcm`, `ncr`, `npr`, `is_prime`, `probability`, and a terminal-only `compare`. Later nodes reference earlier outputs as `@n1`. A plan is executable data: it can be gated, executed, witnessed, diffed, and audited.

4.2 Certification stack

- **Gate G-P1..4 (static).** (1) schema/DAG sanity; (2) type and unit flow — units propagate through `scale`, `convert` must leave from the inbound unit, integer-only slots enforced; (3) ask-target — the final node must produce the asked quantity in the asked unit (the asked unit is parsed from the final sentence, skipping attributive uses such as "the speed in m/s of a truck"); (4) stated-number honesty — every number the question states must be consumed, and no number absent from the question may appear, up to a whitelist of structural constants ({0, 1, 2, 0.5, 100, 1000, 9.8, 3.6, 60} and integers 0–6) that formulas legitimately introduce.
- **Dual execution (dynamic).** The sympy-side executor (`chainexec17`) runs the plan; an mpmath-only witness (`chain_witness17`), sharing no computational code, replays it with depth-budgeted tolerance rel_tol(k)=min(1e-6·3^(k-1), 1e-3). Disagreement at any step or the final value refuses the serve. Refuse-never-crash containment (audit family B27) wraps both.
- **Admission gauntlet for minted content.** Three model-mintable classes — parse reflexes, chain reflexes (pattern + plan template), and connectors (unit adapters, a class proposed by the project owner) — face chained probation on held-out traffic: ≥3 end-witnessed correct serves, ≥2 distinct operand tuples, and a single wrong serve kills the candidate.

4.3 Serving cascade

Chain-marked text (result references, sequential connectives, multi-sentence structure) bypasses the single-step V1.6 engine entirely — a structural rule adopted after audit entry B30 (Section 9.1): **a partial answer to a chain is a wrong answer**. The cascade is then: deterministic *composer* (role-aware text→plan parser) → minted parse/chain reflexes (B31 rule: a chain pattern must consume the entire question) → connector repair → plan-template replay → escalation (UNKNOWN). Every serve, regardless of path, passes the same gate and dual witness.

5 Experimental setup

Corpus (Q-1M). 1,000,000 generated chain questions (seed 1700), committed in-repo with SHA-256 raw-byte hashes: 60.6% non-monotonous phrasing (sub-questions reordered against computation order), 71.7% multi-domain chains (e.g., calculus → finance → physics), 6,000 foreign-phrased chains authored by bridge agents that never see engine internals (isolation rule B9), and 3,624 narrative distractors that must never be answered. Depth mix k=2:20%, k=3:30%, k=4:25%, k=5:15%, k=6:10%. Splits: ACQ 200k (minting offers), DEV 100k (tuning permitted), SEAL 300k (run once), REPLAY 400k (reuse economics).

Arms. `det`: composer + frozen V1.6 library (271 reflexes) — no V1.7 model content. `M-<model>`: det + that model's gauntlet-admitted reflexes. `R-<model>`: det + minted reflexes + that model's dev-phase plan templates, evaluated over the full million (4 x 1M = 4,000,000 evaluations). Planning campaigns (S-PLAN): each model structures 600 dev + 1,200 sealed questions drawn uniformly from det's escalations, one item at a time in serial 25-item batches; the harness measures inter-file write times (first item of each batch excluded as agent startup).

Models. claude-haiku, claude-sonnet, claude-opus, and claude-fable, accessed as sandboxed bridge agents (no raw API access; each agent sees only its own batch file). Engine runs on one Windows 11 workstation, Python 3.12; engine-side cost is 2.9–7.4 ms per query including all gating and witnessing.

6 Results

6.1 Reflexes compose: deterministic chains (COHESION)

On the sealed 300,000 (evaluated once): **112,170 served / 112,170 correct / 0 wrong**; 0 of 1,095 narrative distractors certified; in-domain coverage 37.53%; mean engine cost 2.945 ms/query. On the pre-registered 3+-step stratum: 72,793 of 239,299 = **30.42%** against a frozen 25% bar. Depth decay is graceful (Figure 2): each additional step retains roughly 0.62–0.77 of the previous depth's coverage; V0.9's hard wall (0/25 beyond five steps) is absent — k=6 chains serve at 17.4% with end-to-end witnessing.

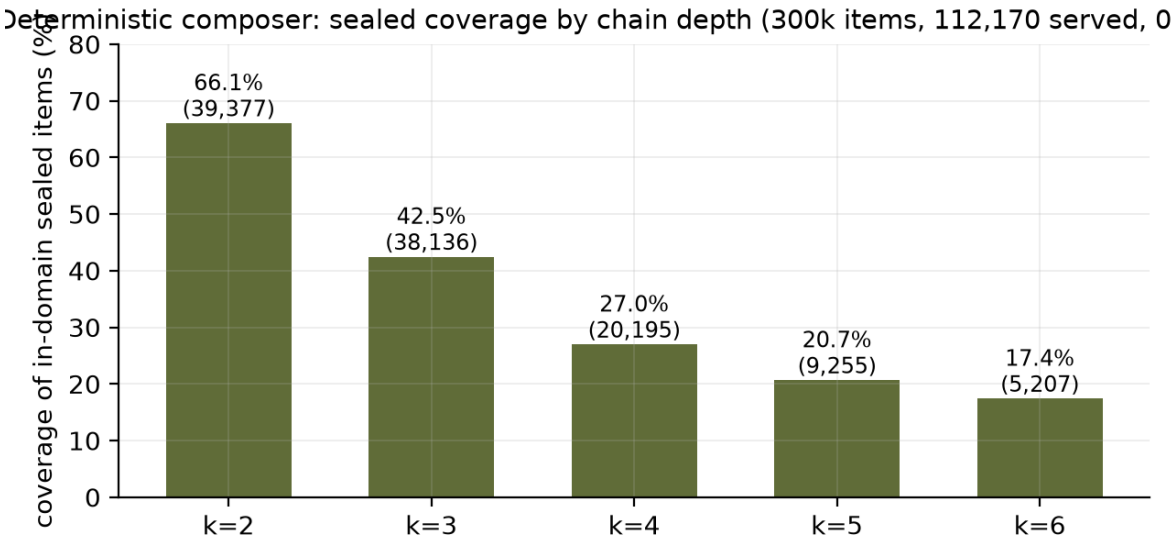


Figure 2: Deterministic sealed coverage by chain depth (bar labels: coverage %, absolute serves). Denominators (in-domain items): k2 59,606; k3 89,782; k4 74,842; k5 44,774; k6 29,901.

6.2 Models plan chains: 4,800 sealed certified structures

Each model structured 1,200 sealed questions the composer had escalated — by construction, questions the deterministic system could *not* answer. Table 1 reports certification against the full gate + dual witness. Wilson 95% intervals on the certified rate are ±1.6–1.8pp.

model	certified	rate (95% CI)	gate-killed	exec-refused	certified-WRONG	dev wrongs (n=600)
haiku	1,077	89.8% (88.0–91.3)	81	29	13	0
sonnet	1,118	93.2% (91.6–94.5)	66	16	0	0
fable	1,138	94.8% (93.4–95.9)	62	0	0	0
opus	1,150	95.8% (94.5–96.8)	50	0	0	0

Table 1: Sealed planning campaign (1,200 chains per model). “Certified” = passed G-P1..4, executed, and independently witnessed; “wrong” = certified yet incorrect against gold.

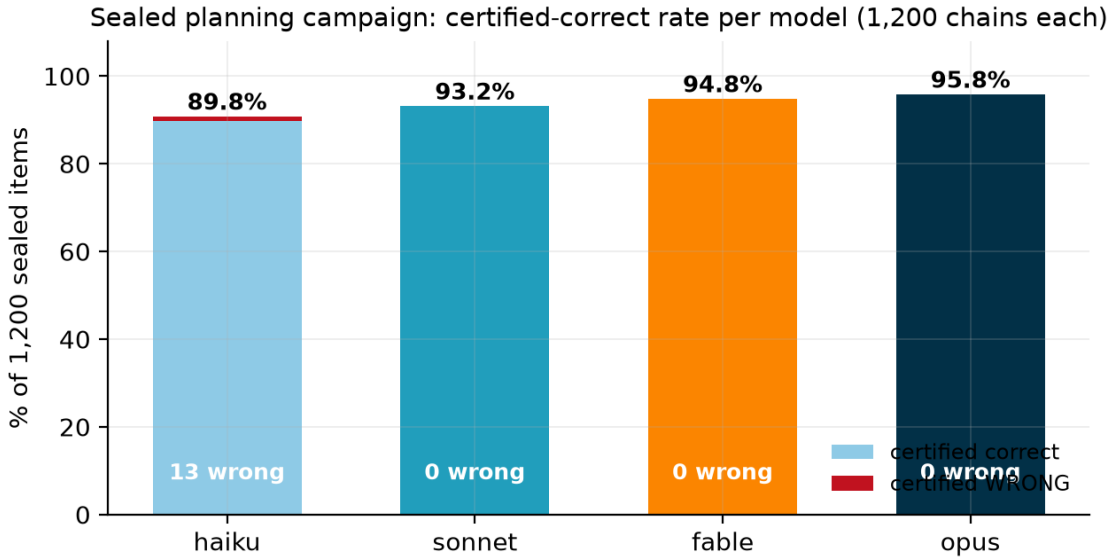


Figure 3: Certified-correct rate per model; the red sliver on haiku is its 13 certified-wrong plans (Section 9.3).

Two observations. First, the three larger models produced **3,406 certified answers on 3,600 sealed chains with zero wrong** — on the exact stratum the deterministic system fails. Second, the dev column is a methods result in itself: 600-item dev campaigns showed *zero* wrongs for **all four** models; haiku's 13 appeared only at sealed volume. Exact binomial: if haiku's true wrong-rate among serves is 1.19% (13/1,090), the probability that a 545-serve dev campaign shows zero is ~0.15% — dev was not merely small, it was *structurally* unable to sample the dual-role collisions involved (Section 9.3). Rare failure modes require sealed volume, not dev iteration.

6.3 How long does structure-finding take?

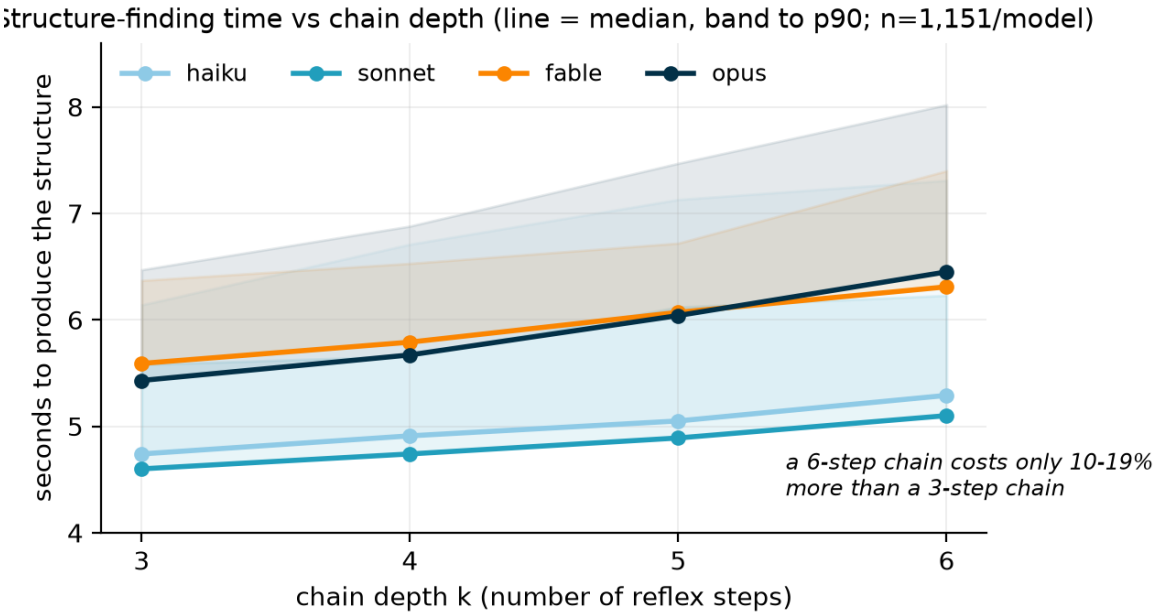


Figure 4: Median structure-finding seconds per chain vs depth (band to p90). Harness-observed inter-file write times; batch-first items excluded; n=1,151/1,200 measured per model (95.9%, above the pre-registered 90% completeness threshold).

model	k=3 med (p90)	k=4 med (p90)	k=5 med (p90)	k=6 med (p90)	k3→k6
sonnet	4.60 (5.58)	4.74 (5.66)	4.89 (6.12)	5.10 (6.23)	+10.9%

model	k=3 med (p90)	k=4 med (p90)	k=5 med (p90)	k=6 med (p90)	k3→k6
haiku	4.74 (6.14)	4.91 (6.71)	5.05 (7.13)	5.29 (7.31)	+11.6%
opus	5.43 (6.47)	5.67 (6.88)	6.04 (7.47)	6.45 (8.02)	+18.8%
fable	5.59 (6.37)	5.79 (6.53)	6.07 (6.72)	6.31 (7.40)	+12.9%

Table 2: Structure-finding time (seconds). Per-depth sample sizes: 349–354 (k3), 370–375 (k4), 245–249 (k5), 177–180 (k6) per model.

• **Depth is nearly free.** Doubling chain depth costs +0.50 to +1.02 s (+10.9–18.8%). The models evidently apprehend the chain structure in one reading rather than deriving it stepwise — consistent with the flat token budget of writing a 6-node vs 3-node DAG dominating over “thinking depth”.

• **Speed anti-correlates with safety.** The fastest planners (sonnet 4.60 s, haiku 4.74 s at k=3) bracket the only unsafe one; the slowest at depth (opus, 6.45 s at k=6) is the most accurate at zero wrong. Fable pairs opus-class safety with a tighter p90 tail (6.72 s vs haiku's 7.13 s at k=5).

• **Absolute scale.** ~5 s of frontier-model time buys a *certified* novel multi-step structure; the engine then serves it in milliseconds. The economics of amortizing that 5 s across repeats is the subject of the next two subsections — and of the failure analysis.

6.4 Million-scale arms and the scale gradient

Each R-arm evaluated all 1,000,000 items with the model's admitted reflexes and dev plan templates layered over the composer. Table 3 gives sealed-split results; totals across all splits appear in Section 9.2. **Zero uncontained engine errors occurred in the 4,000,000 evaluations** (pre-registered SCALE criterion).

arm	served (sealed)	of which template	wrong	wrong-fire	coverage	error / template serve
R-haiku	117,182	5,012	937	0.800%	39.20%	18.70%
R-sonnet	133,177	21,007	840	0.631%	44.55%	4.00%
R-fable	141,048	28,878	542	0.384%	47.19%	1.88%
R-opus	145,918	33,748	357	0.245%	48.82%	1.06%

Table 3: Sealed 300k results per million arm. “Template” serves come from replayed dev plans; the remainder is the composer + minted reflexes, which were error-free in every arm.

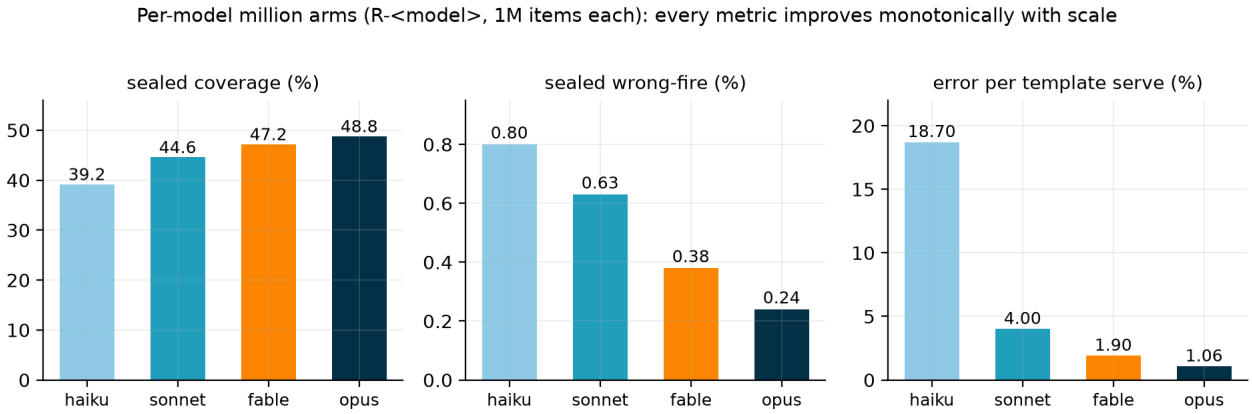


Figure 5: The scale gradient. Coverage rises and both error measures fall strictly monotonically with model scale. V1.1's feared “scale inversion” (bigger miners minting more wrongs) remains falsified one architectural layer up: larger models write templates that are simultaneously more general and cleaner, and opus's gauntlet had zero wrong-kills because it end-anchored every pattern unprompted.

6.5 What planning adds, and reuse economics

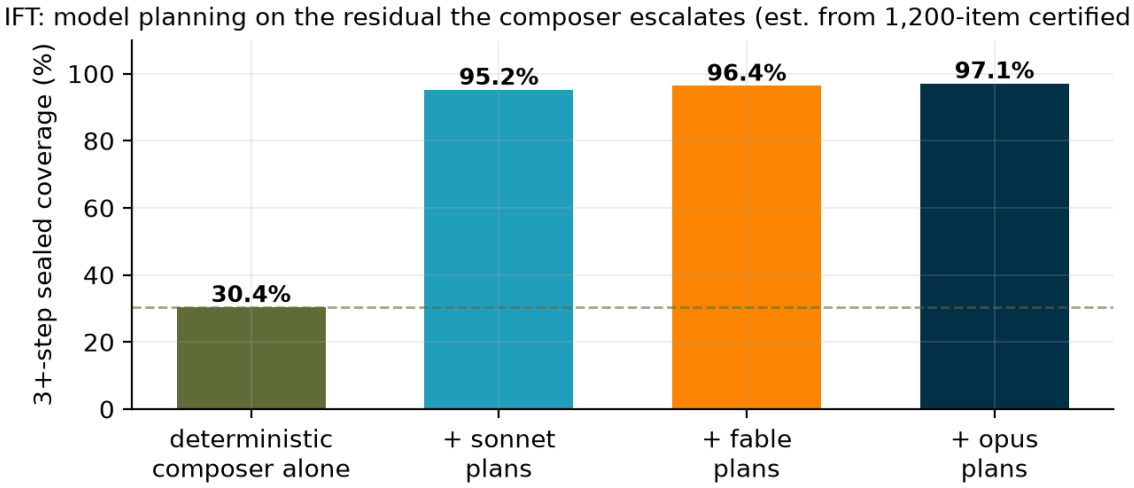


Figure 6: Coverage of the 3+-step sealed stratum. Model bars are estimates: certified rate on the 1,200-item uniform sample of the 166,506-item escalation pool (sampling half-width ±1.2pp at 95%), at zero certified-wrong for the three models shown.

The composer alone covers 30.4% of 3+-step sealed questions; composer + certified planning reaches an estimated **95.2% (sonnet), 96.4% (fable), 97.1% (opus)** — a two-thirds jump (+64.8 to +66.7pp) at zero wrong. haiku is excluded by the frozen SAFETY-3 rule (one certified-wrong plan disqualifies the arm). Reuse mass on

the 400k replay split: each admitted library entry served on average haiku 12.5x, sonnet 48.2x, fable 65.7x, opus 74.9x (pre-registered amortization bar: ≥2x). One ~5-second planning act thus amortizes across dozens of milli-second serves — *if* reuse is safe, which Section 9 shows it currently is not.

6.6 Free-hand minting and an honest null

model	candidates proposed	chain reflexes admitted	gauntlet wrong-kills	connectors proposed	connectors admitted
haiku	(1 chunk unparseable)	0	—	—	0
sonnet	140	37	10	8	0
fable	131	52	18	10	0
opus	151	60	0	12	0

Table 4: Free-hand minting ledger (post-B31 re-gauntlet). Union library: 149 chain reflexes, re-gauntleted jointly to catch cross-model conflicts.

The connector class — unit adapters bridging two reflexes, hypothesized by the project owner — is an **honest null**: all four models proposed connectors (8–12 each) and every one was inert in probation, because the composer already performs final-unit conversion. Free-hand invention found no gap where one was hypothesized; we report the null rather than omit it.

7 Skill cohesion: the arc-level achievement

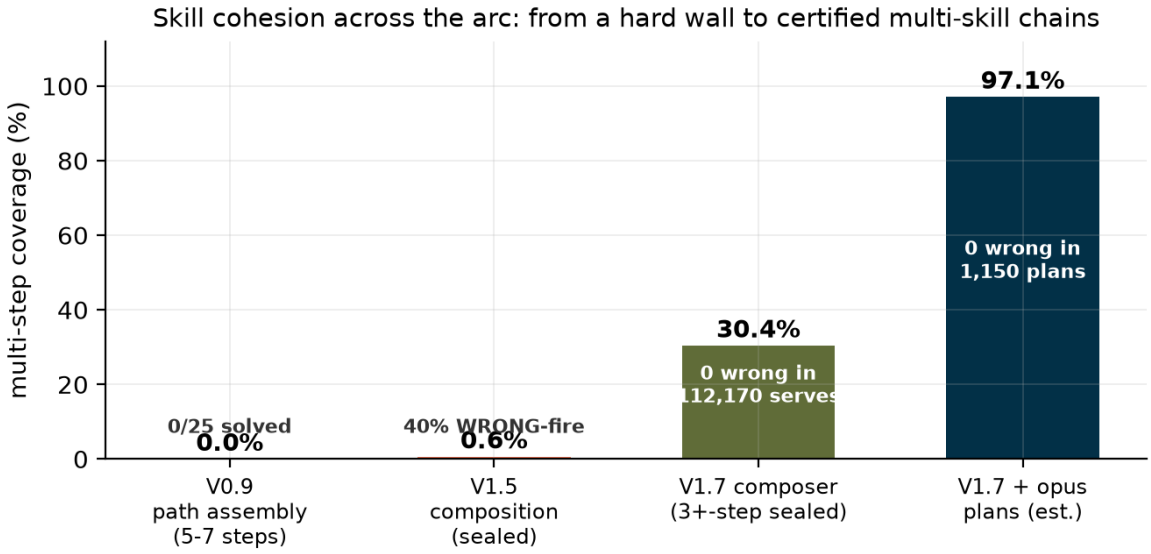


Figure 7: Multi-step capability across the arc. V0.9: 0/25 beyond five steps. V1.5: 0.63% coverage at 40% wrong-fire when composition was forced through an LLM-described state. V1.7: 30.4% deterministic at zero wrong in 112,170 serves; ~97% with certified opus planning at zero wrong.

What changed between V1.5 and V1.7 is the *location of composition*. V1.5 asked a model to describe a problem and guards to filter compositions — and measured 40% wrong-fire, because guards encode which compositions are wrong, never which is right. V1.7 moved composition into typed, executable structure: chains are plans; plans are data; data can be gated by role-aware rules and executed twice independently. The skills a model teaches the engine in isolation (differentiation, lcm, interest, unit conversion — V1.6's 271 reflexes across a 47-leaf taxonomy) now *cohere*: the sealed corpus is 71.7% cross-domain, so nearly every served chain exercises skills from different branches of the taxonomy in one certified computation.

Cohesion also holds across the learning channel itself: the same gauntlet that admits a single-step parse reflex admits a chain reflex; the same gate and witness that certify a composer plan certify a model-planned one; and a certified plan's structure signature makes it a candidate skill for reuse. The stack is one pipeline, not four.

8 The energy position, stated exactly

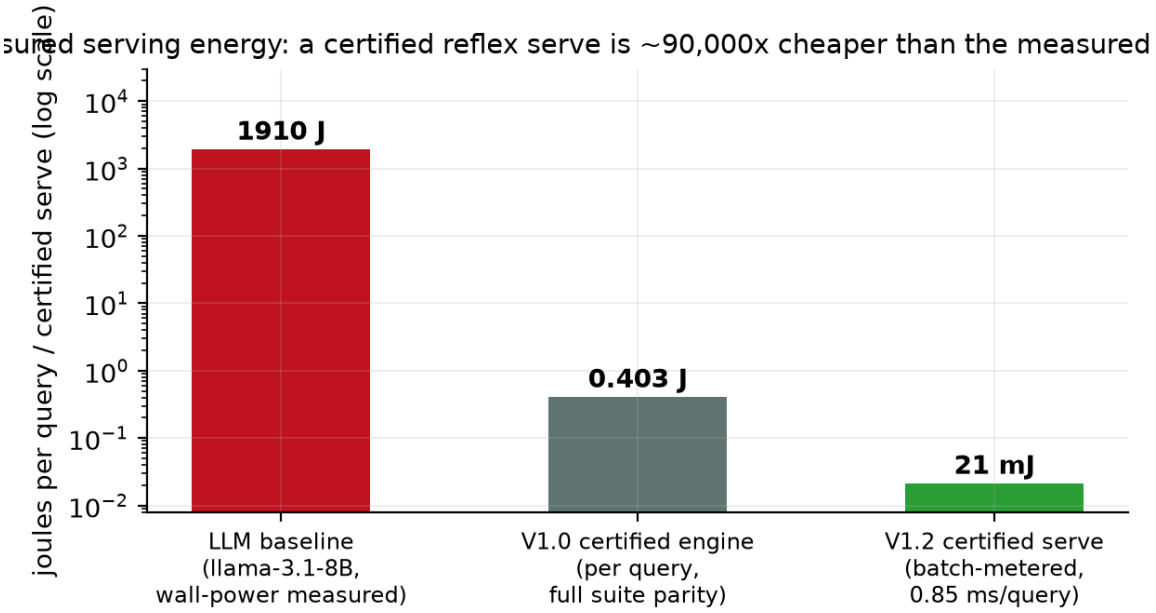


Figure 8: Measured serving-side energy of the certified engine (log scale), wall-power measured on the project workstation. The V1.2 value (21 mJ/certified serve) was batch-metered after audit B24 forced positive controls on the instrument itself and exposed an earlier 1.93 J figure as contamination. Earlier general-purpose configurations that placed an LLM parser/worker inside the serving or state-production loop are out of scope since the V1.6 pivot to the scientific register and are not charted.

What is proven. Serving a certified reflex costs ~21 mJ and ~0.85 ms (V1.2, batch-metered); the V1.7 chain engine runs 2.9–7.4 ms/query CPU-side including double execution and gating. The measured local-LLM baseline (llama-3.1-8B on the same workstation, V1.0/K3 instrumentation) is 1,910 J/query — a serving-energy ratio of roughly **9 x 10^4**. At V1.1's simulated commercial mix, engine-first serving yielded a measured 95.4% fleet-energy saving with 99.7% of residual energy concentrated in the 15.8% of queries that escalated.

Scope of the claim. These figures describe the scientific-register configuration the programme has pursued since the V1.6 pivot, in which **no model is ever in the serving loop** — all 4,000,000 V1.7 evaluations completed with zero model calls; frontier models act only offline, as proposers of procedures and plans whose cost amortizes across serves. No end-to-end fleet energy measurement of the V1.7 configuration itself has been made, so no fleet-level number is claimed here; it is registered future work.

The amortization argument, quantified but conditional. One 5-second planning act (order 10^2–10^3 J of datacenter inference) that certifies a reusable structure serving 48–75 times (Section 6.5) would amortize to per-serve costs competitive with the reflex path — *conditional on safe reuse*, which Section 9 shows requires role-aware binding first. We decline to book the energy win before the safety prerequisite exists.

9 Failure analysis: 8,842 wrong answers, fully traced

9.1 B30 — the boundary failure

On first contact with merged chain traffic, the frozen V1.6 single-step parser answered the *first sub-question* of 5 chains (dev 3 of 37,570 serves = 0.008%; ACQ 2 of 75,219 = 0.003%) — e.g., serving 52 = 4x13 for a four-step question beginning “Compute 4*(6+7)…”. The end-to-end witness cannot catch this: a first-step serve executes a* valid single-step plan*; only the gold disagrees. Fix (structural, no V1.6 module touched): chain-marked text is out of the single-step engine's domain by definition. Post-fix reruns: zero recurrence in ~1.1M dev/ACQ evaluations.

9.2 B31 — the prefix failure

One minted sonnet reflex (`chain_lcm_interest`) lacked an end anchor and matched the two-step *prefix* of four-step questions: 67 wrong dev serves from a single reflex (all 67 of M-sonnet's dev wrongs). Its 2,000-offer probation never sampled a colliding longer chain; dev's 100k did — the same volume lesson as Section 6.2. Fix: whole-question consumption (match must end within 2 characters of the stripped text) enforced in the gauntlet *and* the serving path, closing the class for every candidate rather than the one instance. Post-fix: all dev arms zero wrong; union arm (M-all) covered 49.6% of dev at zero wrong.

9.3 B32 — positional reuse is structurally unsafe (principal finding)

Dev-certified plans were carried into the R-arms as *templates*: question numbers became regex capture groups; plan structure was retained. Across 4M evaluations this layer produced every single wrong answer of the version:

arm	ACQ 200k	DEV 100k	SEAL 300k	REPLAY 400k	total
R-haiku	605	289	937	1,238	3,069
R-sonnet	555	261	840	1,112	2,768
R-fable	345	167	542	708	1,762
R-opus	256	108	357	522	1,243
sum	1,761	825	2,676	3,580	8,842

Table 5: Template-replay wrong answers by arm and split. In the identical runs, the composer path re-served its 112,170 sealed answers with zero errors in every arm.

Row-level tracing shows one failure family with three mechanisms:

- **Donor-constant swap.** Whitelisted template literals travel with the template. Traced example (sealed q17_0346197): template minted from a chain ending “multiply by 3” matched a question whose numbers differ only in whitelist-small constants; served 2.16, gold 6.48 — exactly the dropped x3. The honesty gate is blind here *by construction*: the whitelist that prevents value-smuggling exempts precisely these constants.
- **Role misbinding.** Capture groups bind numbers that all appear in the question — into the wrong roles. Traced example (dev q17_0285930): an integration bound captured as [1,1] gives integral 0, force = 0.0 served, gold 6,191.0. Every literal was legitimately present; only the *roles* were swapped (rate↔years, multiplier↔bound).
- **Dropped dual-role usage** (haiku's 13 certified-wrong *plans*, distinct from templates). When one stated number fills two roles — “multiply by 7 … at 7% per year” — a plan using 7 once satisfies magnitude-level honesty while silently omitting a step. 12 of 13 traced wrongs sit on derivative- or sum-headed chains into interest with exactly this shape.

The fix that worked, and the one that cannot. A strict binding rule (`--strictreplay`: every bound slot value must be stated in the question) was implemented and probed on 5,000 dev items — never on the run-once sealed data. It eliminated the donor-constant mechanism (4 candidates blocked; arm wrong-fire 0.73%→0.46%) and **9 wrongs survived, all role misbindings**. No literal-presence filter can see roles; the information simply is not in the check.

In 4,000,000 evaluations, no wrong answer ever came from a role-aware component, and every wrong answer came from a role-blind one. The composer (role-aware parser): 0 wrong in 412,000+ serves across all

arms and splits. Fresh certified plans by the three larger models: 0 wrong in 3,600. Positional templates and magnitude-only honesty: 8,842 + 13. This reduces safe skill reuse from an open problem to a specified mechanism — bind templates by structure signature *and role slots*; extend stated-number honesty from magnitudes to role-tagged usages (which also closes haiku's class).

9.4 Pre-registered verdict

The frozen ladder ranks any safety breach above any capability result. Sealed wrong-fire of 0.800% (R-haiku) and 0.631% (R-sonnet) breached the pre-registered 0.5% ceiling, and haiku's 13 plan wrongs breached SAFETY-3; the version's outcome is therefore **E**, with COHESION, PLAN-LIFT, TIMING, amortization (η), and SCALE all met. We report the letter prominently because the willingness to lose it honestly is what makes the zero-wrong claims elsewhere credible.

10 Threats to validity and limitations

• **Synthetic corpus.** Q-1M is generator-produced. Non-monotonous phrasing (60.6%), cross-domain chaining (71.7%), foreign-phrased items authored by isolated agents, and in-stream narrative distractors mitigate — but do not eliminate — shared-grammar effects between the composer and the corpus. The arc's real-language record (V1.3: 69.7% coverage at 1.44% wrong on GSM8K-class text; V1.2: 519/520 abstentions zero-shot) shows natural language remains a distinct, harder frontier. Claims here are about chain *composition and certification*, not about open natural language.

• **Coverage ceilings.** Deterministic coverage is 37.5% overall and 17.4% at k=6; most deep traffic still escalates. Model planning covers the residual only at ~5 s per novel structure.

• **PLAN-LIFT is an extrapolation.** Stratum coverage with planning is estimated from a 1,200-item uniform sample of the escalation pool (±1.2pp); a full sealed planning pass over 166,506 items was not run.

• **Timing includes harness overhead.** Inter-file write times bundle model latency with agent-harness turnaround; the measure is comparative across models and depths under an identical harness, not an absolute inference-latency claim.

• **Energy values predate V1.7.** Figure 8's engine measurements were made in V1.0/V1.2 on the same workstation; the V1.7 chain configuration itself has no end-to-end energy measurement yet, and none is claimed.

• **Single harness, single operator.** All experiments ran on one machine under one experimenter's protocol; pre-registration and in-repo artifacts (hashes, wrong-row ledgers, superseded runs, 4,800 raw timed plan files) are the audit trail offered in lieu of independent replication, which we would welcome.

11 Conclusions and the road to the goal

V1.7 establishes that certified skills compose. A deterministic engine chained up to six independently witnessed computational steps through 112,170 sealed serves without a single error; three frontier models structured the questions it could not — 3,600 sealed chains — likewise without a single wrong certified answer, in about five seconds per structure nearly independent of depth. Every failure in four million evaluations localized to one specified mechanism: role-blind binding in the reuse layer.

Against the programme goal — **AI scientific solvers that consume orders of magnitude less energy and whose learned skills cohere** — the standing is: serving energy is measured at ~10^-2 J against ~10^3 J for an LLM baseline (proven at the component level, unproven fleet-wide for V1.7); skill cohesion has moved from a wall (V0.9), through an unsafe attempt (V1.5, 40% wrong-fire), to certified chains at scale; and the learning channel admits skills from four different frontier models through one gauntlet with quality monotone in model scale.

V1.8 is therefore an engineering task with a falsifiable bet, to be pre-registered like the nineteen before it: **role-aware reuse** — templates matched by structure signature and role slots rather than positional regex, and gate honesty extended from magnitudes to role-tagged usages — with the prediction that the 8,842-wrong class vanishes at unchanged coverage; followed by an end-to-end energy measurement of the lean configuration. If the bet fails, that too will be reported.

Reproducibility. Repo `reflex-engine`, tag `v1.7-final` (819115d); frozen snapshot `reflex-engine-v1.7-frozen`. Q-1M with SHA-256 hashes, all arm summaries, row-level wrong ledgers, mint ledgers, the 4,800 raw sealed plan files (modification times preserved in `results/v17/v17_plan_archive.zip`), control batteries, and the numbered bug audit (`bug_audit_v1_7.md`) are committed. Single-command repro per arm: `python v17_run.py --split seal --arm det`; `python v17_run.py --split <s> --arm R-<m> --mintlib results/v17/stack_<m>.json`; `python tools/planner17.py score --model <m> --phase sealed`. Prior-version reports (V1.0–V1.6, MD+PDF) are in the same repository.